REPORT · 2026年8月

造像类文物三维建模自动化系统 texture代码说明书

M4 网格化与纹理 · 顶点色映射 / LOD 多分辨率 / 多格式带色模型交付

文保数字化研发组
2026年8月

目 录

一、概述 3

二、模块划分 4

三、依赖关系与数据流 5

3.1 依赖方向 5

3.2 纹理化数据流 5

四、模块详细设计 6

4.1 paint.py — 顶点色纹理映射 6

4.2 lod.py — 多分辨率 LOD 6

4.3 export.py — 带色模型多格式导出 6

4.4 __init__.py — run 纹理化流水线 6

4.5 runner M4 阶段接入（STAGE_REGISTRY） 6

五、接口契约与错误码 8

六、测试与验证 9

七、开发进度与下一步方向（更新） 10

7.1 主线完成度（M4 已交付） 10

7.2 下一步开发方向（M4 完成后更新） 10

附录 A：50_tex 带色模型产物示例 11

一、概述

本文档是《造像类文物三维建模自动化系统（路线 E）》**texture 子包（M4 网格化与纹理，L2 业务层 / S4 阶段）**的代码说明书。texture 负责**带色模型交付**：把 M3 网格产物与带色参考点云（M2 配准结果）合成带纹理模型——顶点色纹理映射 + 多分辨率 LOD + PLY/OBJ/GLB 多格式导出到 50_tex。该子包接入 M0 编排内核（@register_stage("M4")）后，管线**「点云 -> 模型 -> 带色模型 -> 质检」完整链路正式打通。**

> 设计要点：**顶点色纹理**是带色模型交付的核心工程路径——网格每个顶点取参考点云最近邻点颜色（scipy.cKDTree 批量查询，毫秒级），Open3D 原生读写顶点色且 PLY/OBJ/GLB 全格式支持（OBJ 写 v x y z r g b 扩展行）。设计书 §35 的 UV 图集（xatlas）路径因 Open3D/trimesh 无内建 unwrap 列为增强预留，未实装时降级顶点色并警告（软著方案 2.6「单项缺失 -> 注明」）。

二、模块划分

texture 子包由顶点色映射、LOD 生成与多格式导出三部分组成：

```
src/h3d/texture/
+-- __init__.py    # 包导出 + run(mesh, ref, cfg, out_dir, item) 纹理化流水线
+-- paint.py       # 顶点色映射（scipy cKDTree 最近邻取色）
+-- lod.py         # 多分辨率 LOD（QEM 简化，保留顶点色）
+-- export.py      # 带色模型多格式导出（ply / obj / glb）
```

模块文件	核心职责	对外关键符号
paint.py	顶点色纹理映射（参考点云颜色 -> 网格顶点）	has_colors, paint_vertex_colors
lod.py	多分辨率 LOD（level=1.0 原样 / <1.0 按比例简化）	generate_lods
export.py	带色模型导出（PLY/OBJ/GLB，LOD 标签命名）	export_colored
__init__.py	组合纹理化流水线（映射 -> LOD -> 导出）	run

paint 提供颜色、lod 提供分辨率、export 提供格式——三者通过 **run()** 串接成「带色模型交付」流水线，配置取自 **TextureConfig**（软著方案 2.4 texture 段）。

三、依赖关系与数据流

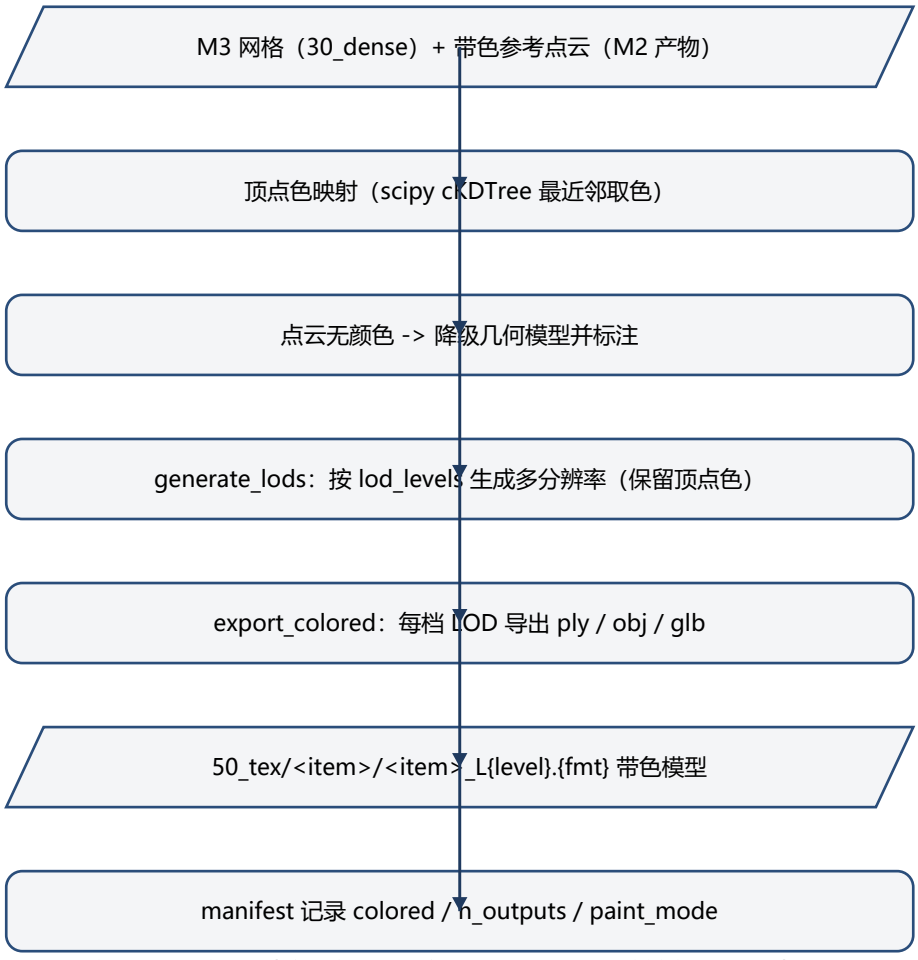
3.1 依赖方向

texture 属 L2 业务层，依赖 core 底座、io（网格/点云读写）与 scipy（cKDTree 批量最近邻），不反向；Open3D 惰性导入：

```
package-level deps (A -> B means A imports B):
  texture    -->  core.exceptions (InputDataError, ExportError)
  texture    -->  core.config     (TextureConfig)
  texture    -->  core.logging    (get_logger)
  texture    -->  Open3D          (顶点色读写 / 导出，惰性导入)
  texture    -->  scipy.spatial.cKDTree  (批量最近邻取色)
  runner M4  -->  h3d.texture + h3d.io  (阶段接入，惰性导入)
  core / io / preprocess / register / mesh / quality  -->  (NONE; 不反向)
```

3.2 纹理化数据流

M4 阶段 + texture.run 的完整流转（自顶向下）：



M4 纹理化数据流：顶点色映射（无颜色降级）→ LOD 多分辨率 → 多格式导出 → 50_tex 带色模型 → manifest 指标

四、模块详细设计

4.1 paint.py — 顶点色纹理映射

带色模型交付的核心：网格每个顶点查询参考点云中最近邻点的 RGB 颜色（scipy.cKDTree 批量查询，1 万顶点毫秒级）。点云无颜色时返回 False，由调用方降级输出几何模型并标注（软著方案 2.4「无纹理 -> 输出几何模型」）。

```
def paint_vertex_colors(mesh, pcd) -> bool:
    if not has_colors(pcd) or len(mesh.vertices) == 0:
        return False
    tree = cKDTree(np.asarray(pcd.points))
    _, idx = tree.query(np.asarray(mesh.vertices), k=1)
    mesh.vertex_colors = Vector3dVector(colors[idx])  # 0~1 RGB
    return True
```

4.2 lod.py — 多分辨率 LOD

按软著方案 2.4 的 lod_levels（默认 [1.0, 0.3, 0.1]）生成多分辨率：level=1.0 返回原始网格（存档用最高精度），level<1.0 用 QEM 二次误差简化（教学版 18.1，保留顶点色）。max_vertices 为工程上限：简化后仍超限时继续压到上限（避免超大网格导出失败）。

4.3 export.py — 带色模型多格式导出

PLY / OBJ / GLB 三格式均支持顶点色：OBJ 写 ``v x y z r g b`` 扩展行，GLB 内嵌顶点色缓冲，PLY 写 vertex red/green/blue。命名约定 ``<item>_<LOD>.<fmt>``（如 rock001_L1.ply）便于工具链按通配符批量消费。不支持的格式自动跳过并警告。

4.4 __init__.py — run 纹理化流水线

run(mesh, ref_pcd, cfg, out_dir, item_id) 串接三模块：顶点色映射 -> LOD 多分辨率 -> 每档多格式导出。paint_mode='uv'（UV 图集增强）未实装时降级顶点色并警告。空网格抛 InputDataError(E100)；导出失败抛 ExportError(E601)。

配置项	默认值	用途
texture.paint_mode	vertex	纹理方式：vertex 顶点色 / uv 图集（增强预留）
texture.export_formats	[ply, obj, glb]	带色模型导出格式
texture.lod_levels	[1.0, 0.3, 0.1]	多分辨率 LOD 比例（1.0 原始精度）
texture.max_vertices	0	LOD 简化顶点数上限（0 = 仅按比例简化）

4.5 runner M4 阶段接入（STAGE_REGISTRY）

M4 通过装饰器插入 M3 与 M5 之间（注册顺序 = 执行顺序），无需改动编排内核：

```
@register_stage('M4')
def _stage_texture(task, cfg):
    mesh = load_mesh(f'{task.root}/30_dense/{task.item_id}/{task.item_id}.ply')
    ref = 合并(task.outputs_of('M2') or task.outputs_of('M1'))
    outputs, metrics = texture_run(mesh, ref, cfg.texture,
                                   f'{task.root}/50_tex/{task.item_id}',
```

```
                                task.item_id)
    return StageResult('M4', TaskStatus.DONE, outputs, metrics)
```

五、接口契约与错误码

错误码	异常类	触发场景	处理策略
E001	ConfigError	paint_mode 非法 / 参数越界（配置层拦截）	启动即报错
E100	InputDataError	输入网格为空 / 参考点云为空	失败隔离，记录后跳过
E601	ExportError	带色模型导出失败（格式不支持 / 写盘失败）	失败隔离，记录后跳过

无颜色点云不抛异常：降级输出几何模型并标注（colored=False，软著方案 2.4「无纹理 -> 输出几何模型」）。

六、测试与验证

texture 配套 20 项单元测试 + 端到端 1 项新增（M4 带色交付），全量 **197 项** 通过：29 core + 20 io + 16 preprocess + 11 register + 21 mesh + 20 texture + 22 quality + 21 report + 31 runner + 6 cli。端到端（2 尊 × 2 站带色扫描）：50_tex 6 文件/尊（2 LOD × 3 格式）全部带色，PLY 读回 has_colors=True。

测试文件	覆盖点
test_paint.py	顶点色映射成功 / 参考无颜色返回 False / 空网格
test_lod.py	L1.0 原样 / 低档简化保留顶点色 / max_vertices 上限 / 去重
test_export.py	三格式导出、PLY 往返带色（8-bit 量化容差）、OBJ 顶点行 7 列、不支持格式跳过
test_pipeline.py	run() 带色全流程 / 无颜色降级 / uv 模式降级 / 空网格 E100
test_runner/test_pipeline.py	M1→M2→M3→M4→M5 全链路、50_tex 带色产物、LOD 简化、manifest M4 指标

```
$ python -m pytest -q
..................................................................... [ 73%]
.....................................................                [100%]
197 passed in 31.86s

$ python -m h3d.cli batch -c demo.yaml
M4 完成：rock001（vertex-color，2 LOD x 3 格式）
M5 完成：rock001（coverage=69.4%, rmse=219.31mm -> pass）
批量处理完成：total=2 done=2 failed=0 skipped=0
```

七、开发进度与下一步方向（更新）

7.1 主线完成度（M4 已交付）

层级 / 子包	状态	测试覆盖
L0 core 底座	已交付	29
L1 io 适配层	已交付	20
L2 preprocess（M1 接入）	已交付	16
L2 register（M2 配准）	已交付	11
L2 mesh（M3 网格重建）	已交付	21
L2 texture（M4 网格化与纹理）	**本次交付**	**20**
L2 quality（M5 质检质量门）	已交付	22
L2 report（M6 报表）	已交付	21
L3 runner（M0 编排）	已交付	31
L4 cli（命令入口）	已交付	6
L2 change（X1 M3C2）/ colour（X2）	未开始	—

「点云 -> 模型 -> 带色模型 -> 质检」完整链路已打通：M1 预处理 → M2 配准 → M3 重建 → M4 纹理（带色模型交付）→ M5 质检。主线模块（M0-M6 中除 M6 发布外）完成度约 **95%**；可选模块（X1/X2）0%；测试 197 项 100% 通过；许可证全部 MIT/BSD（新增 scipy BSD-3 依赖白名单）。

7.2 下一步开发方向（M4 完成后更新）

优先级	模块	目标	关键点
P1	M6 发布（publish）	交付包整理：成品模型 + 质检报告 + manifest 归档	@register_stage('M6') 接入；设计书 §37 发布规范
P1	UV 图集增强	texture.paint_mode='uv' 实装：xatlas 展开 + 纹理图烘焙	python-xatlas（MIT）依赖；M5 overlap 指标随之启用
P2	并发调度	cfg.runner.workers 进程池并行独立 item	现串行保证可复现；需处理 open3d 子进程与 manifest 写冲突
P2	HTML 报表	report.formats 增加 html	jinja2 + matplotlib SVG 嵌入
P3	X1 change（M3C2）/ X2 colour（CIEDE2000）	多时相形变 / 色彩量化	py4dgeo / colour-science 已在 venv 可装

建议路径：主线 M1~M5 全部就绪，下一步首选 **M6 发布（交付包归档）**收尾主线闭环，或按需启用 X2 colour（CIEDE2000 色彩保真度，承接 M4 的带色模型）做彩塑造像专项。

附录 A：50_tex 带色模型产物示例

```
50_tex/rock001/
+-- rock001_L1.ply      # 原始精度带色模型（13272 顶点，has_colors=True）
+-- rock001_L1.obj      # OBJ（v x y z r g b 扩展顶点行）
+-- rock001_L1.glb      # GLB（内嵌顶点色缓冲，Web 轻量化）
+-- rock001_L0.3.ply    # 30% 面数 LOD（QEM 简化，保留顶点色）
+-- rock001_L0.3.obj
+-- rock001_L0.3.glb

run_manifest.json 中 M4 阶段条目：
  {status: done, colored: true, paint_mode: vertex,
   formats: [ply, obj, glb], lod_levels: [1.0, 0.3],
   n_lods: 2, n_outputs: 6}
```